Trailblazer CBL | Advanced Topics

1. Game and Level Design

Why we chose this topic

We chose level design as our topic because level design plays a huge role on game being entertaining because game is basically a puzzle game with various levels. Make them too easy and people would get bored, make them too hard people get frustrated and leave.

What we learned from the research

Reading Tim Ryan's Beginning Level Design, Part 1 helped us think about levels from the player's perspective. He says to connect challenge to the games central skills, stresses the value of simple rules and variation, and warns against frustrating or unsolvable situations. For our game we interpret this as testing route planning and the order of robot activation. More obstacles alone do not necessarily create a better puzzle. Their placement should make the player reason about the mechanics.

How this appears in our project

The project contains four separate maps. The first has one robot and one matching plate, while the fourth contains four robots and four plates. This creates a progression feeling by first getting player familiar by the game. Obstacles restrict available paths, and the same arrow mechanic can produce different puzzles through changes in placement. The introduction explains the rules and restart and solution controls make retrying a puzzle and inspecting a route possible.

Challenges, decisions and what we learned

The main design challenge is that a map can be valid without being a good puzzle. The code checks structural conditions, including a player, an exit and matching robot and plate colours, but these checks do not prove that a solution exists or that the difficulty is appropriate. Therefore we spent some time tuning our games levels to the extent that they are not that easy and not that challenging. The saved solution layouts provide useful reference routes.

Tim Ryan. *Beginning Level Design, Part 1*. Game Developer, 16 April 1999. gamedeveloper.com/design/beginning-level-design-part-1

2. Git Branching Workflows

Why we chose this topic

We chose git branching and workflows because this project required working in pairs and each of us had our own tasks. We had a way to combine contributions while having a history of what changed.

What we learned from the research

Atlassian's comparison explains that a workflow should suit the team and that published approaches can be adapted rather than followed as fixed rules. We took from this that our process should remain simple enough to use consistently. We had multiple branches to allow work to proceed separately, while merging brings it into a shared version.

How this appears in our project

Our repository uses a shared main branch and a development branch. The history records pull-request merges from that branch alongside changes to robot movement, menus, selectable levels and the finished game. There are also direct commits to main, so we would describe the actual process as a mixture rather than a blind implementation of one named workflow. We used both branches accordingly so we could recover quickly if there was a mistake

Challenges, decisions and what we learned

Some pushes were rejected because GitHub already contained newer commits.The solution we come up with at the end was to preserve the backlog commit and merge the newer game changes; a safety backup retained the interrupted working-file changes. This experience helped us distinguish a saved local commit from a successfully published change. It also showed that document locks and local file state can affect version-control work even when the game code is correct.

Impact and improvements for future work

The workflow gives our project a shared version and a traceable record of development. For future work, we might use more short-lived branches named after specific tasks like a separate branch for level design, make focused commits and synchronise before starting and before merging. We would also agree on a small pre-merge check, such as launching the game and checking the affected level. These are improvements that we could use for our future work.

Atlassian. *Comparing Git workflows: What you should know*. Atlassian Git Tutorials. atlassian.com/en/git/tutorials/comparing-workflows